CSET419 – Introduction to Generative AI

Lab – 11

Objective

The objective of this lab is to fine-tune a pre-trained generative model (GPT-2) for real-world applications. Students will fine-tune GPT-2 to build a Product Review Generator for e-commerce and a Recipe Instruction Generator for a food-tech application, learning how transfer learning adapts a general model to specific business domains.

Learning Outcomes

After completing this lab, students will be able to:

1. Understand how fine-tuning applies to real-world industry applications
2. Load and configure a pre-trained GPT-2 model using Hugging Face Transformers
3. Prepare real-world domain-specific datasets for causal language modeling
4. Fine-tune the model and compare generated output before and after training
5. Evaluate the practical quality of generated product reviews and recipe instructions

What is Fine-Tuning for Real-World Applications?

Fine-tuning takes a pre-trained model that already understands general language and trains it further on domain-specific data. This is how companies build AI-powered features without training models from scratch.

Real-world examples of fine-tuning:

- **E-Commerce:** Generate product descriptions, customer reviews, and recommendation text
- **Food-Tech:** Generate recipe instructions, meal plans, and cooking tips from ingredient lists
- **Healthcare:** Generate clinical notes, patient summaries, and medical documentation
- **Customer Support:** Generate context-aware chatbot replies trained on company FAQ data

- **Marketing:** Generate ad copy, email campaigns, and social media posts in brand voice

Experiment:

Component–I: Fine-Tune GPT-2 as a Product Review Generator (E-Commerce)

Scenario: You are an AI engineer at an e-commerce company. The company wants to build an AI tool that auto-generates realistic product reviews to help sellers understand what good reviews look like and to populate demo storefronts. Your task is to fine-tune GPT-2 on real product review data.

Tasks

1. Load GPT-2 and generate baseline product reviews (before fine-tuning)
2. Prepare the product review dataset and tokenize it
3. Fine-tune the model on product review data
4. Generate product reviews from the fine-tuned model and compare with baseline

Dataset

Use the following product review corpus:

```
this phone has an amazing battery life and the camera quality is outstanding for the price.
i bought this laptop for college and it handles all my assignments and coding projects perfectly.
the sound quality of these headphones is incredible with deep bass and clear vocals.
this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.
great wireless earbuds with noise cancellation that blocks out all background sound.

the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.
this portable charger saved me during travel and it charges my phone three times on a single charge.
the tablet screen is bright and colorful which makes watching movies a great experience.
i love this fitness tracker because it motivates me to reach my daily exercise goals.
this bluetooth speaker is compact but delivers surprisingly loud and clear audio.

the delivery was fast and the product was packed securely with no damage at all.
excellent value for money and the build quality feels premium despite the affordable price.
the customer service team was very helpful when i had questions about the product features.
this camera takes stunning photos in low light and the video recording quality is very smooth.
i have been using this product for three months and it still works perfectly like day one.
```

the design is sleek and modern and it looks great on my desk next to my other gadgets.
easy to set up right out of the box and the instructions were clear and simple to follow.
highly recommend this product to anyone looking for quality and reliability at a fair price.
the software updates keep adding new features which makes this purchase even more worthwhile.
best purchase i made this year and i would definitely buy from this brand again.

Code

Step 1: Setup and Load Model

```
!pip install transformers datasets accelerate -q

import torch, math
from transformers import (GPT2LMHeadModel, GPT2Tokenizer, Trainer,
    TrainingArguments, DataCollatorForLanguageModeling, set_seed)
from datasets import Dataset
set_seed(42)

tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id
```

Step 2: Generate Baseline Reviews (Before Fine-Tuning)

```
def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors='pt').to(model.device)
    with torch.no_grad():
        out = model.generate(inputs, max_length=max_length, temperature=0.8,
            top_k=50, do_sample=True, pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0], skip_special_tokens=True)

review_prompts = [
    'This product is',
    'I bought this phone and',
    'The quality of this item',
]

print('=== BASELINE REVIEWS (Before Fine-Tuning) ===')
baseline = {}
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f'Prompt: {p}\nOutput: {baseline[p]}\n')
```

Step 3: Prepare Dataset and Fine-Tune

```
corpus = [
    'this phone has an amazing battery life and the camera quality is
outstanding for the price.',
```

```
    'i bought this laptop for college and it handles all my assignments and
coding projects perfectly.',
    'the sound quality of these headphones is incredible with deep bass and
clear vocals.',
    'this smartwatch tracks my steps accurately and the heart rate monitor is
very reliable.',
    'great wireless earbuds with noise cancellation that blocks out all
background sound.',
    'the keyboard feels very comfortable for long typing sessions and the
backlight is a nice touch.',
    'this portable charger saved me during travel and it charges my phone
three times on a single charge.',
    'the tablet screen is bright and colorful which makes watching movies a
great experience.',
    'i love this fitness tracker because it motivates me to reach my daily
exercise goals.',
    'this bluetooth speaker is compact but delivers surprisingly loud and
clear audio.',
    'the delivery was fast and the product was packed securely with no damage
at all.',
    'excellent value for money and the build quality feels premium despite the
affordable price.',
    'the customer service team was very helpful when i had questions about the
product features.',
    'this camera takes stunning photos in low light and the video recording
quality is very smooth.',
    'i have been using this product for three months and it still works
perfectly like day one.',
    'the design is sleek and modern and it looks great on my desk next to my
other gadgets.',
    'easy to set up right out of the box and the instructions were clear and
simple to follow.',
    'highly recommend this product to anyone looking for quality and
reliability at a fair price.',
    'the software updates keep adding new features which makes this purchase
even more worthwhile.',
    'best purchase i made this year and i would definitely buy from this brand
again.',
]

dataset = Dataset.from_dict({'text': corpus})
tokenized = dataset.map(lambda x: tokenizer(x['text'], truncation=True,
    max_length=128, padding='max_length'), batched=True,
remove_columns=['text'])
split = tokenized.train_test_split(test_size=0.15, seed=42)

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
mlm=False)

training_args = TrainingArguments(
    output_dir='./gpt2-reviews', num_train_epochs=15,
    per_device_train_batch_size=4, learning_rate=5e-5,
    weight_decay=0.01, warmup_steps=50, eval_strategy='epoch',
    logging_steps=10, save_strategy='no',
    fp16=torch.cuda.is_available(),
)
```

```
trainer = Trainer(model=model, args=training_args,
    train_dataset=split['train'], eval_dataset=split['test'],
    data_collator=data_collator)

trainer.train()
```

Step 4: Generate Reviews and Compare

```
eval_res = trainer.evaluate()
print(f'Perplexity: {math.exp(eval_res["eval_loss"]):.2f}')

print('\n=== FINE-TUNED REVIEWS (After Fine-Tuning) ===')
for p in review_prompts:
    ft_out = generate_text(model, tokenizer, p)
    print(f'Prompt: {p}')
    print(f'  Baseline:   {baseline[p][:120]}')
    print(f'  Fine-Tuned: {ft_out[:120]}\n')
```

<u>Expected Output</u>

- Baseline generates random generic text (news, Wikipedia-style content)
- Fine-tuned model generates realistic product review language (e.g., "great value", "highly recommend", "battery life is amazing")
- Clear shift in tone, vocabulary, and structure toward e-commerce review style

Component–II: Fine-Tune GPT-2 as a Recipe Instruction Generator (Food-Tech)

Scenario: You are an AI developer at a food-tech startup. The company is building a smart cooking app that suggests recipes. Your task is to fine-tune GPT-2 to generate cooking instructions when given a dish name as a prompt.

Tasks

1. Use the same model setup from Component – I (reload a fresh GPT-2)
2. Prepare the recipe instruction dataset and tokenize it
3. Fine-tune the model on recipe data
4. Generate recipe instructions using dish name prompts
5. Compare baseline vs. fine-tuned recipe outputs

Dataset

Use the following recipe instruction corpus:

to make butter chicken start by marinating chicken pieces in yogurt with turmeric chili powder and garam masala for one hour.
heat butter in a pan and fry onions until golden brown then add ginger garlic paste and cook for two minutes.
add tomato puree and cook on low heat for ten minutes until the oil separates from the masala.
add the marinated chicken and cook on medium heat for fifteen minutes until fully cooked.
finish with fresh cream and kasuri methi and serve hot with naan or steamed rice.

for pasta carbonara boil spaghetti in salted water until al dente and reserve half cup of pasta water.
fry diced pancetta in olive oil until crispy and set aside.
whisk together egg yolks parmesan cheese and black pepper in a bowl.
toss the hot pasta with pancetta and remove from heat then quickly stir in the egg mixture.
the residual heat will cook the eggs into a creamy sauce and serve immediately with extra parmesan.

to prepare vegetable stir fry heat sesame oil in a wok on high heat.
add sliced bell peppers broccoli florets and snap peas and toss for three minutes.
pour in soy sauce oyster sauce and a pinch of sugar and stir well.
add minced garlic and ginger and cook for one more minute until fragrant.
serve the stir fry over steamed jasmine rice and garnish with sesame seeds.

for chocolate chip cookies cream together butter and sugar until light and fluffy.
beat in eggs one at a time then add vanilla extract and mix well.
fold in flour baking soda and salt then gently stir in chocolate chips.
scoop dough onto a baking sheet and bake at 180 degrees for twelve minutes until golden.

let cookies cool on the tray for five minutes before transferring to a wire rack.

Code

Step 1: Reload Fresh Model

```
# Load a fresh GPT-2 for this component
tokenizer2 = GPT2Tokenizer.from_pretrained('gpt2')
model2 = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id
```

Step 2: Generate Baseline Recipes (Before Fine-Tuning)

```
recipe_prompts = [
    'To make butter chicken',
    'For pasta carbonara',
    'To prepare a chocolate cake',
]

print('=== BASELINE RECIPES (Before Fine-Tuning) ===')
baseline2 = {}
for p in recipe_prompts:
    baseline2[p] = generate_text(model2, tokenizer2, p)
    print(f'Prompt: {p}\nOutput: {baseline2[p]}\n')
```

Step 3: Prepare Recipe Dataset and Fine-Tune

```
recipes = [
    'to make butter chicken start by marinating chicken pieces in yogurt with
turmeric chili powder and garam masala for one hour.',
    'heat butter in a pan and fry onions until golden brown then add ginger
garlic paste and cook for two minutes.',
    'add tomato puree and cook on low heat for ten minutes until the oil
separates from the masala.',
    'add the marinated chicken and cook on medium heat for fifteen minutes
until fully cooked.',
    'finish with fresh cream and kasuri methi and serve hot with naan or
steamed rice.',
    'for pasta carbonara boil spaghetti in salted water until al dente and
reserve half cup of pasta water.',
    'fry diced pancetta in olive oil until crispy and set aside.',
    'whisk together egg yolks parmesan cheese and black pepper in a bowl.',
    'toss the hot pasta with pancetta and remove from heat then quickly stir
in the egg mixture.',
    'the residual heat will cook the eggs into a creamy sauce and serve
immediately with extra parmesan.',
    'to prepare vegetable stir fry heat sesame oil in a wok on high heat.',
    'add sliced bell peppers broccoli florets and snap peas and toss for three
minutes.',
    'pour in soy sauce oyster sauce and a pinch of sugar and stir well.',
    'add minced garlic and ginger and cook for one more minute until
fragrant.',
```

```
    'serve the stir fry over steamed jasmine rice and garnish with sesame
seeds.',
    'for chocolate chip cookies cream together butter and sugar until light
and fluffy.',
    'beat in eggs one at a time then add vanilla extract and mix well.',
    'fold in flour baking soda and salt then gently stir in chocolate chips.',
    'scoop dough onto a baking sheet and bake at 180 degrees for twelve
minutes until golden.',
    'let cookies cool on the tray for five minutes before transferring to a
wire rack.',
]

dataset2 = Dataset.from_dict({'text': recipes})
tok2 = dataset2.map(lambda x: tokenizer2(x['text'], truncation=True,
    max_length=128, padding='max_length'), batched=True,
remove_columns=['text'])
split2 = tok2.train_test_split(test_size=0.15, seed=42)

collator2 = DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)

args2 = TrainingArguments(
    output_dir='./gpt2-recipes', num_train_epochs=15,
    per_device_train_batch_size=4, learning_rate=5e-5,
    weight_decay=0.01, warmup_steps=50, eval_strategy='epoch',
    logging_steps=10, save_strategy='no',
    fp16=torch.cuda.is_available(),
)

trainer2 = Trainer(model=model2, args=args2,
    train_dataset=split2['train'], eval_dataset=split2['test'],
    data_collator=collator2)

trainer2.train()
```

Step 4: Generate Recipes and Compare

```
eval2 = trainer2.evaluate()
print(f'Perplexity: {math.exp(eval2["eval_loss"]):.2f}')

print('\n=== FINE-TUNED RECIPES (After Fine-Tuning) ===')
for p in recipe_prompts:
    ft = generate_text(model2, tokenizer2, p)
    print(f'Prompt: {p}')
    print(f'  Baseline:   {baseline2[p][:120]}')
    print(f'  Fine-Tuned: {ft[:120]}\n')
```

<u>Expected Output</u>

- Baseline generates unrelated text (news, Wikipedia-style) for cooking prompts
- Fine-tuned model generates step-by-step cooking instructions with ingredients, temperatures, and timing

- Generated recipes follow a realistic cooking flow (marinate → cook → serve)

Lightweight & Free (Best for Colab/Lab) Can use in place of GPT-2:

Model	Size	Why Use It
DistilGPT-2	82M	Even smaller than GPT-2, trains faster, great for quick labs
GPT-Neo (125M/1.3B)	125M – 1.3B	Open-source GPT-3 alternative by EleutherAI
GPT-J (6B)	6B	Stronger generation, needs LoRA/QLoRA on Colab
Phi-2 / Phi-3-mini	2.7B / 3.8B	Microsoft's small but powerful models
TinyLlama (1.1B)	1.1B	LLaMA architecture, very efficient, runs on free Colab
LLaMA 3.2 (1B/3B)	1B – 3B	Meta's latest small models, excellent quality
Gemma 2 (2B)	2B	Google's lightweight open model
Qwen2.5 (0.5B/1.5B)	0.5B – 1.5B	Alibaba's small models, very capable
BLOOM (560M)	560M	Multilingual, great for non-English tasks
OPT (125M/350M)	125M – 350M	Meta's open GPT alternative, easy to fine-tune